École d'Ingénierie Informatique

BTC1 – Sécurité des Systèmes d'Information

Rapport d'Audit Complet
Sécurité des Systèmes d'Information

Projet AEGIS – Infrastructure TechSud

Auteurs – Groupe 3 :

Adje Kenneth
Divine Merveille
Davian Mbayi
Toure Souwoye

Formation :

Groupe BTC1-IPSSI
Avril 2026

Dépôt GitHub : `Aegis-groupeX`

Table des matières

1 Contexte et Objectifs du Projet AEGIS **2**

2 Fondements Théoriques de la Sécurité Appliquée **2**
2.1 Algorithmes de chiffrement et de hachage 2
2.2 Du stockage naïf au *salting* 3

3 Déploiement de l'Infrastructure Initiale **3**
3.1 Création de la machine virtuelle (VirtualBox) 4
3.2 Récupération du socle de code (Git) 4
3.3 Séparation des privilèges : création du compte administrateur 5

4 Durcissement des Composants (*Hardening*) **5**
4.1 Sécurisation du service SSH 5
4.2 Filtrage réseau avec UFW 7
4.3 Analyse comportementale via Fail2ban 7

5 Versioning et Arborescence GitHub **8**

6 Script d'Audit Automatisé (Python) **10**

7 Tableau de Vulnérabilités Synthétique **12**

8 Conclusion et Conformité RGPD **12**

1. Contexte et Objectifs du Projet AEGIS

TechSud, une PME fictive en pleine croissance, a récemment subi une compromission majeure de son système d'information. L'attaque a mis en évidence plusieurs faiblesses structurelles critiques : absence de pare-feu réseau, configuration SSH laxiste, manque de surveillance et absence de politique de durcissement (*hardening*).

Le projet AEGIS a été mandaté avec les objectifs suivants :

— **Auditer** l'état initial du serveur pour identifier les vecteurs de compromission.

— **Durcir** l'infrastructure selon les standards de l'industrie (CIS Benchmarks, ANSSI).

— **Mettre en place des outils de protection active** contre les attaques automatisées (bruteforce).

— **Automatiser la validation** via des scripts Bash (déploiement) et Python (audit continu).

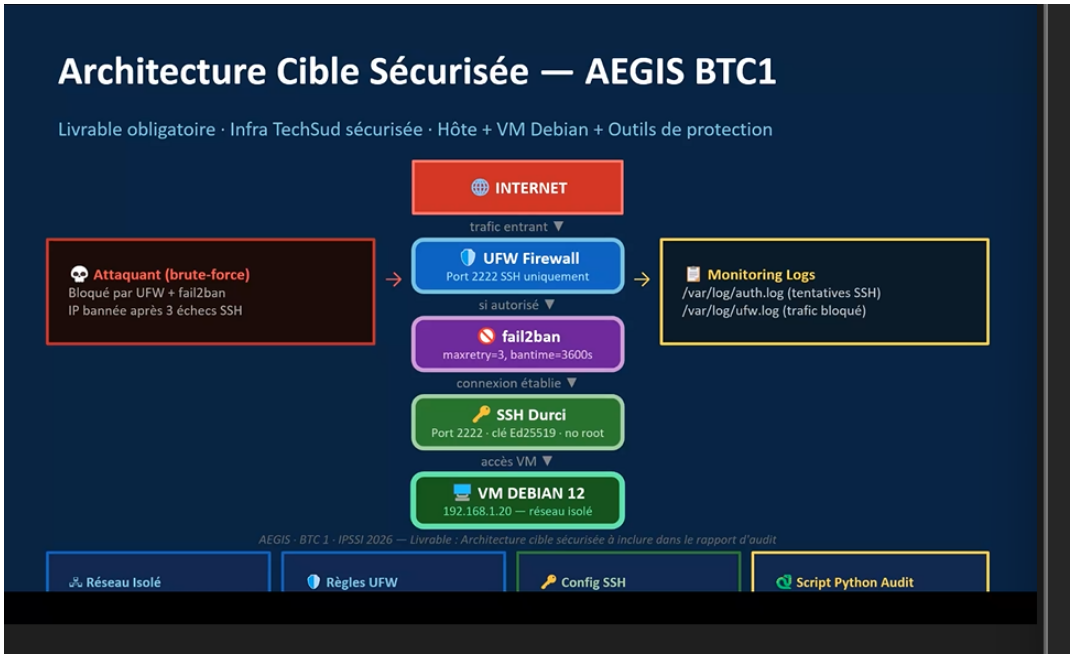

Figure 1. Architecture cible sécurisée – AEGIS BTC1. La défense en profondeur repose sur trois couches complémentaires : UFW pour le filtrage réseau (couche 4), Fail2ban pour l'analyse comportementale des logs, et un accès SSH verrouillé par cryptographie asymétrique.

2. Fondements Théoriques de la Sécurité Appliquée

Avant toute manipulation de l'infrastructure, il est essentiel de justifier nos choix technologiques, notamment concernant l'abandon de l'authentification par mot de passe.

2.1 Algorithmes de chiffrement et de hachage

La robustesse d'un système repose sur des fondations cryptographiques solides. Le choix de l'algorithme détermine la résistance aux attaques par force brute ou par collision.

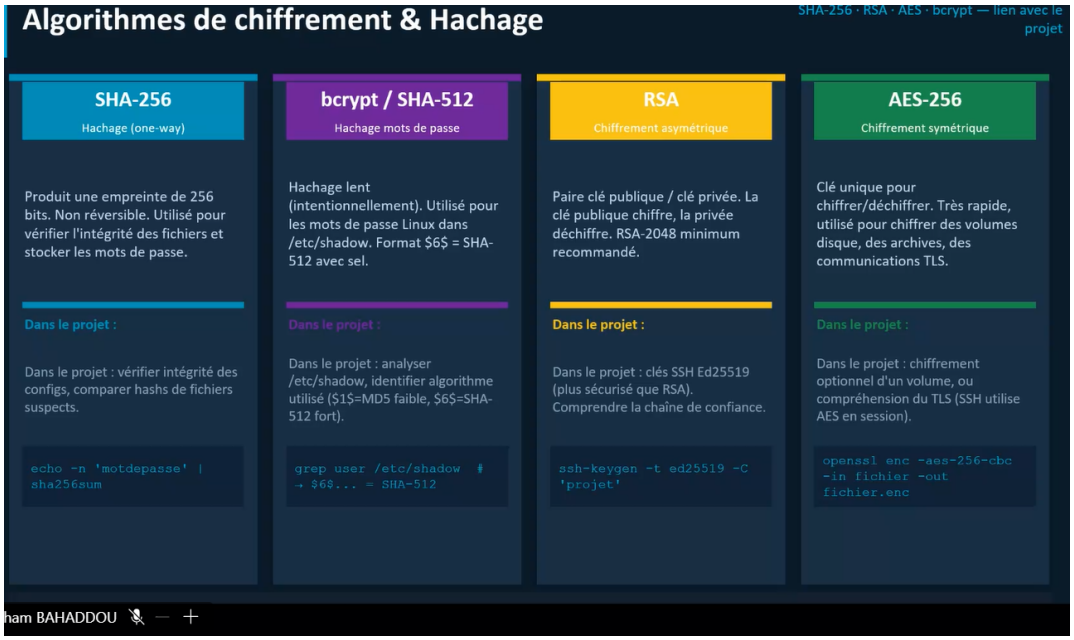

Figure 2. Panorama des algorithmes cryptographiques. Le projet utilise **SHA-512** pour le hachage des secrets locaux (`/etc/shadow`) et **Ed25519** (plus performant que RSA-4096) pour les échanges asymétriques SSH.

2.2 Du stockage naïf au *salting*

Historiquement, les mots de passe étaient stockés en clair, puis hachés avec des algorithmes simples (MD5, SHA-1). Ces approches restent vulnérables aux attaques par dictionnaire et aux *Rainbow Tables*.

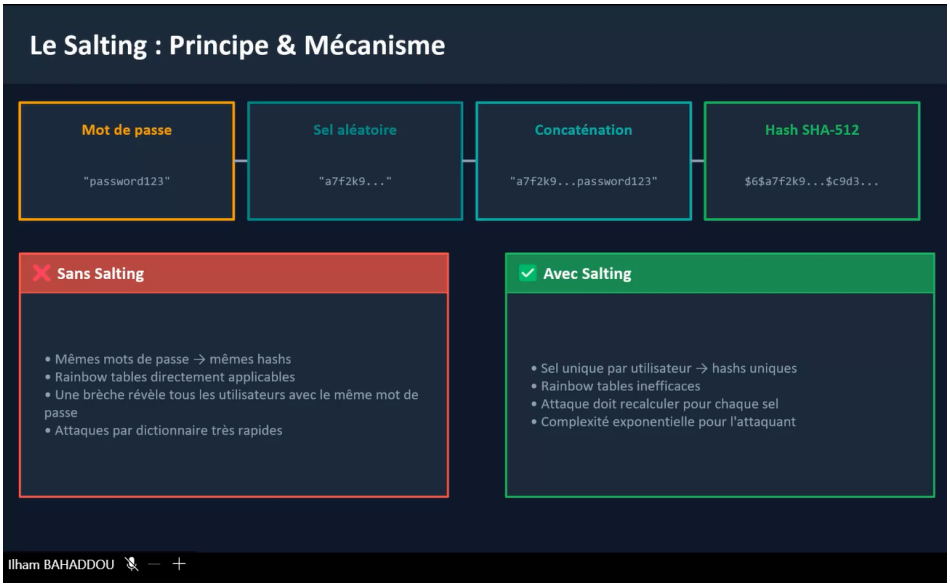

Figure 3. Principe du *salting*. Bien que le sel prévienne les attaques par Rainbow Tables, l'authentification par mot de passe sur un service SSH exposé reste vulnérable à l'ingénierie sociale et au vol de credentials. D'où la transition vers les clés cryptographiques asymétriques.

3. Déploiement de l'Infrastructure Initiale

3.1 Création de la machine virtuelle (VirtualBox)

L'environnement a été recréé localement avec une machine Debian 12 configurée avec deux interfaces réseau : **Host-Only** pour l'administration sécurisée depuis l'hôte, et **NAT** pour les sorties Internet nécessaires à APT et GitHub.

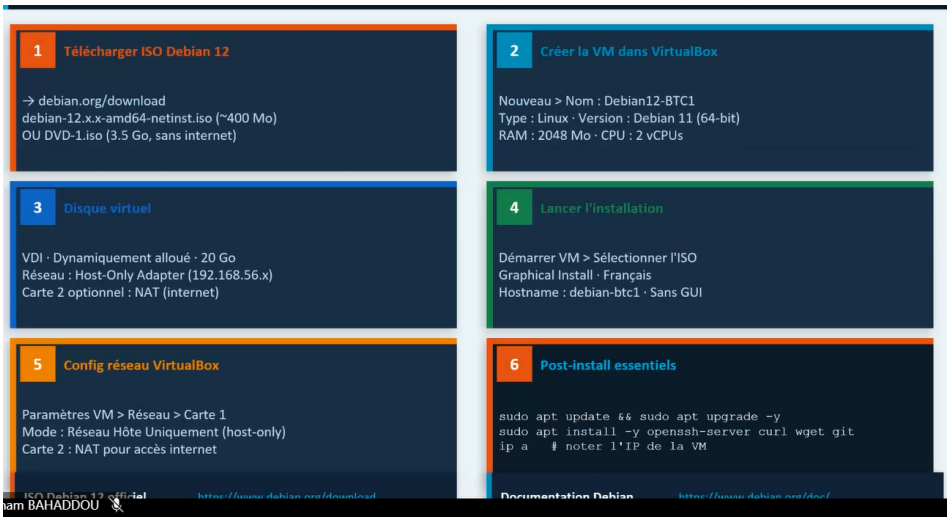

Figure 4. Checklist de configuration de la VM Debian 12 : allocation des ressources et topologie réseau validées.

3.2 Récupération du socle de code (Git)

Une infrastructure moderne se gère via l'*Infrastructure as Code* (IaC). Git a été installé pour rapatrier les scripts depuis le dépôt centralisé.

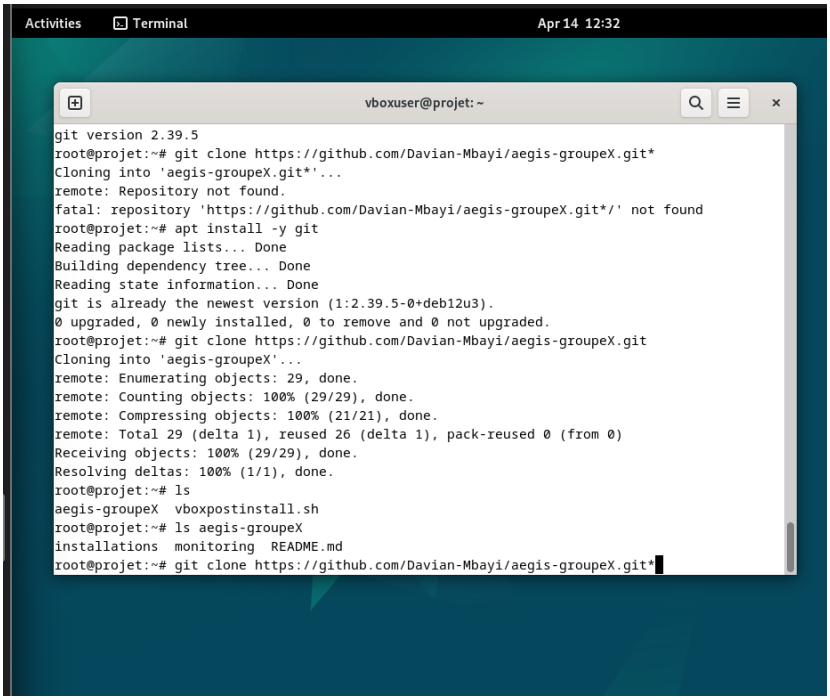

Figure 5. Installation d'APT et clonage du dépôt `aegis-groupeX`. Une erreur typographique mineure sur l'URL a été rapidement corrigée.

3.3 Séparation des privilèges : création du compte administrateur

L'audit initial a révélé l'usage dangereux du compte `root`. L'utilisateur `sousou` a été créé et ajouté au groupe `sudo` selon le principe du moindre privilège.

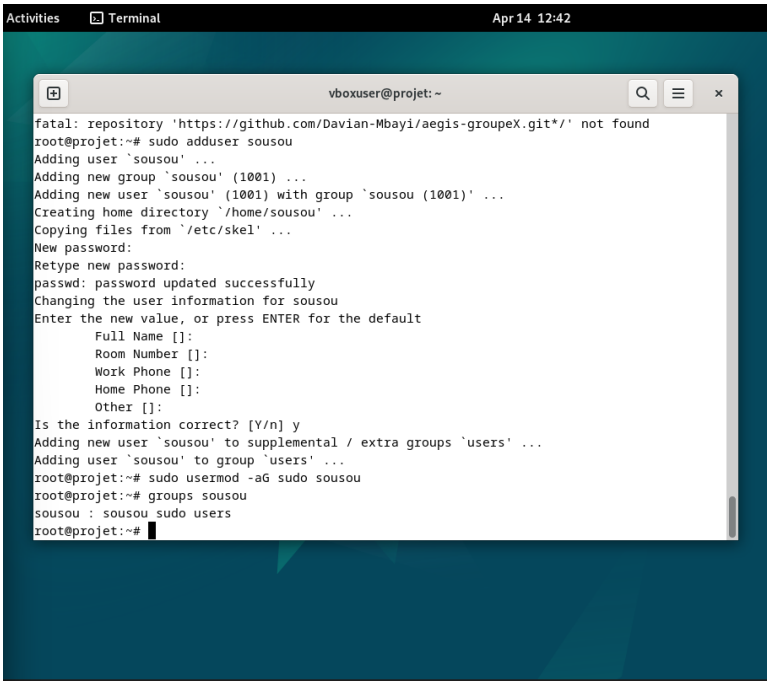

Figure 6. Commande `adduser` pour générer le répertoire `/home` depuis `/etc/skel`, suivie de `usermod -aG sudo` pour accorder les droits d'élévation de privilèges.

4. Durcissement des Composants (*Hardening*)

4.1 Sécurisation du service SSH

Le service OpenSSH a été configuré pour refuser catégoriquement les mots de passe. Une paire de clés `Ed25519` a été générée depuis la machine hôte.

```
C:\Users\SOUWOYE>ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\SOUWOYE/.ssh/id_ed
25519): ssh-copy_id chefsupreme 10.74.2.85
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ssh-copy_id chefsupreme 10
.74.2.85
Your public key has been saved in ssh-copy_id chefsupreme 10.74.
2.85.pub
The key fingerprint is:
SHA256:unGp8a/uc/NpDeUl+QKgFXoVeyh0o6VvI4HMv3xA8x8 souwoye@Souso
u_Star
The key's randomart image is:
+--[ED25519 256]--+
|          o.*.   |
|       o +o* +   |
|        =oO.o .. |
|        .+ *..+ .|
|        S + =+E+ |
|       . o =.oo..|
|      + o o .o.. |
|       B. o....  |
|      oo==.+o    |
+----[SHA256]-----+

C:\Users\SOUWOYE>
```

Figure 7. Génération de la paire de clés Ed25519. Le *randomart* SHA256 fournit une validation visuelle de l'empreinte de la clé.

Après déploiement de la clé publique, les directives suivantes ont été appliquées dans `/etc/ssh/sshd_config` :

```
PermitRootLogin no
PasswordAuthentication no
```

Listing 1. Directives de durcissement SSH

```
C:\Users\SOUWOYE>ssh sousou@10.74.2.85
The authenticity of host '10.74.2.85 (10.74.2.85)' can't be esta
blished.
ED25519 key fingerprint is SHA256:7fil3VncXara5R/hoAhn69T9ZX/NLK
mJ85MNLWUkNQQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprin
t])? yes
Warning: Permanently added '10.74.2.85' (ED25519) to the list of
 known hosts.
sousou@10.74.2.85's password:
Linux projet 6.1.0-44-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.16
4-1 (2026-03-09) x86_64

The programs included with the Debian GNU/Linux system are free
software;
the exact distribution terms for each program are described in t
he
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the exten
t
permitted by applicable law.
sousou@projet:~$
```

Figure 8. Première connexion réussie via la clé Ed25519. L'hôte est ajouté au fichier `known_hosts`, prévenant ainsi les attaques de type Man-in-the-Middle (MitM).

4.2 Filtrage réseau avec UFW

UFW a été configuré pour appliquer le principe du moindre privilège réseau : tout fermer par défaut, n'ouvrir que l'indispensable.

```
sousou@projet:~$ sudo ufw default deny incoming
[sudo] password for sousou:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
sousou@projet:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
sousou@projet:~$
```

Figure 9. Application des politiques globales : `deny incoming` (bloque les connexions non sollicitées) et `allow outgoing` (autorise le serveur à se mettre à jour).

```
sousou@projet:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)

sousou@projet:~$
```

Figure 10. Statut verbose d'UFW : seuls les ports 22 (SSH), 80 (HTTP) et 443 (HTTPS) constituent les points d'entrée autorisés.

4.3 Analyse comportementale via Fail2ban

Fail2ban surveille `/var/log/auth.log` et bannit dynamiquement via iptables les adresses IP présentant plus de 3 échecs de connexion SSH.

Problème rencontré – Compatibilité Debian 12

Debian 12 utilise `systemd-journald` plutôt que `rsyslog` par défaut. Fail2ban n'a pas trouvé le fichier de log SSH, causant un arrêt avec `status=255`.

```
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
sousou@projet:~$ sudo systemctl enable --now fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/systemd/systemd
-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
sousou@projet:~$ sudo systemctl status fail2ban
× fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
     Active: failed (Result: exit-code) since Wed 2026-04-15 13:02:00 CEST; 17s ago
   Duration: 92ms
       Docs: man:fail2ban(1)
    Process: 4174 ExecStart=/usr/bin/fail2ban-server -xf start (code=exited, status=255/E>
   Main PID: 4174 (code=exited, status=255/EXCEPTION)
        CPU: 88ms

Apr 15 13:02:00 projet systemd[1]: Started fail2ban.service - Fail2Ban Service.
Apr 15 13:02:00 projet fail2ban-server[4174]: 2026-04-15 13:02:00,727 fail2ban.configread>
Apr 15 13:02:00 projet fail2ban-server[4174]: 2026-04-15 13:02:00,737 fail2ban           >
Apr 15 13:02:00 projet fail2ban-server[4174]: 2026-04-15 13:02:00,737 fail2ban           >
Apr 15 13:02:00 projet systemd[1]: fail2ban.service: Main process exited, code=exited, st>
Apr 15 13:02:00 projet systemd[1]: fail2ban.service: Failed with result 'exit-code'.
lines 1-15/15 (END)
```

Figure 11. Échec initial du démarrage de Fail2ban (`code=exited, status=255`).

```
sousou@projet:~$ sudo fail2ban-client -t
2026-04-15 13:07:34,097 fail2ban.configreader   [4188]: WARNING 'allowipv6' not defined in
 'Definition'. Using default one: 'auto'
2026-04-15 13:07:34,105 fail2ban                [4188]: ERROR   Failed during configuratio
n: Have not found any log file for sshd jail
2026-04-15 13:07:34,105 fail2ban                [4188]: ERROR   ERROR: test configuration
failed
sousou@projet:~$
```

Figure 12. Diagnostic via `fail2ban-client -t` : le message *"Have not found any log file for sshd jail"* identifie la cause racine. La correction du paramètre `backend` dans `jail.local` a résolu le problème.

5. Versioning et Arborescence GitHub

L'une des forces du projet AEGIS réside dans son architecture logicielle. L'ensemble des configurations a été scripté et centralisé sur GitHub, garantissant la reproductibilité et la traçabilité des opérations.

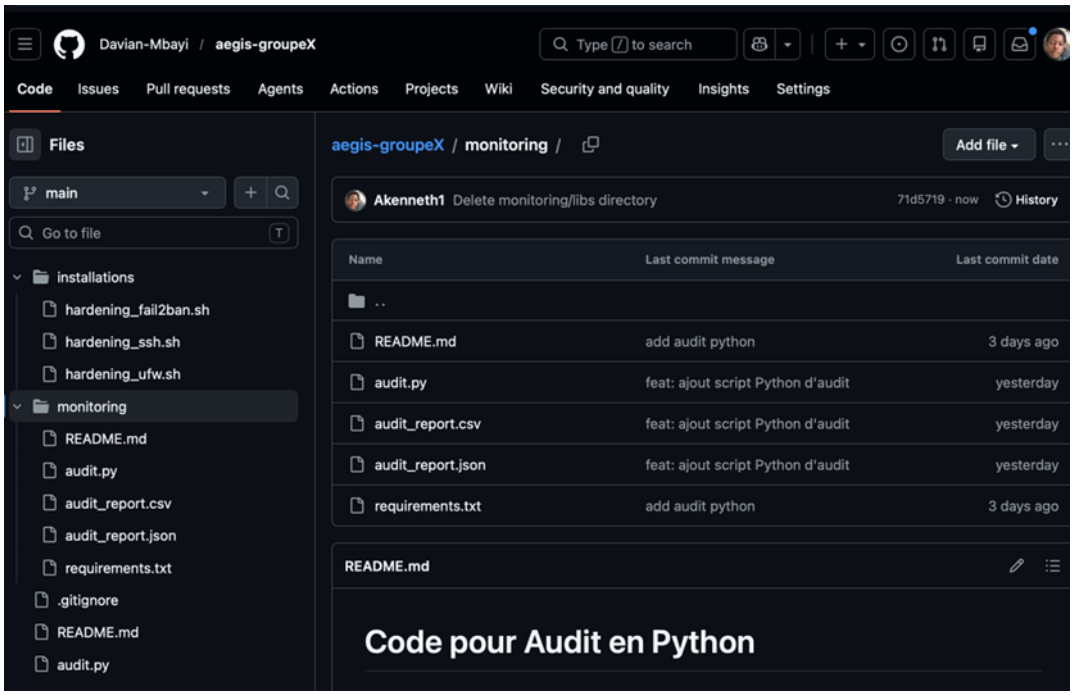

FIGURE 13. Arborescence du dépôt `aegis-groupeX`. La séparation stricte entre scripts de déploiement et outils de surveillance reflète une organisation modulaire.

L'arborescence du dépôt repose sur deux dossiers distincts :

Dossier `installations/`

Contient les scripts Bash modulaires : `hardening_ssh.sh`, `hardening_ufw.sh`, `hardening_fail2ban.sh`. Cette approche atomique permet d'exécuter, de déboguer ou de mettre à jour chaque module de sécurité indépendamment.

Dossier `monitoring/`

Contient le script `audit.py` ainsi que ses fichiers de sortie (`audit_report.csv` et `audit_report.json`), servant de preuves horodatées de la conformité du serveur.

Le déploiement sur une machine vierge s'effectue via de simples commandes `curl` pointant vers la branche `main` du dépôt :

```
sousou@projet:~$ curl -s https://raw.githubusercontent.com/Davian-Mbayi/ae
gis-groupeX/main/installations/hardening_ufw.sh | sudo bash
[*] Installation et configuration du pare-feu UFW...
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ufw is already the newest version (0.36.2-1).
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Firewall is active and enabled on system startup
[OK] UFW configuré.
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
```

Figure 14. Exécution dynamique du script UFW directement depuis la branche `main`, garantissant l'utilisation de la dernière version validée du code.

6. Script d'Audit Automatisé (Python)

Le script Python `audit.py` valide la conformité de la configuration système en interrogeant les ports ouverts, l'état des services et les directives SSH critiques.

```
macbook — kenadmin@AEGIS-DEBIANN: ~/aegis-groupeX/monitoring — ssh kenadmin@10.74.2.84 — 125×31
  GNU nano 7.2                                audit.py
#!/usr/bin/env python3
import subprocess
import json
import csv
from datetime import datetime

def run_cmd(cmd):
    try:
        return subprocess.check_output(cmd, shell=True, text=True).strip()
    except subprocess.CalledProcessError:
        return "ERROR"

def check_ssh_config():
    sshd = run_cmd("grep -E 'PermitRootLogin|PasswordAuthentication' /etc/ssh/sshd_config")
    return sshd

def check_ufw_status():
    return run_cmd("sudo ufw status verbose")

def check_fail2ban_status():
    status = run_cmd("sudo systemctl is-active fail2ban")
    jail = run_cmd("sudo fail2ban-client status sshd")
    return status, jail

def check_open_ports():
    return run_cmd("ss -tulpen")

                                  [ Read 64 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Figure 15. Le Script Python

```
^X
sousou@projet:~$ cat /etc/passwd | grep chefsupreme
chefsupreme:x:1002:1002:,,,:/home/chefsupreme:/bin/bash
sousou@projet:~$ python3 audit.py
python3: can't open file '/home/sousou/audit.py': [Errno 2] No such file o
r directory
sousou@projet:~$ sudo
usage: sudo -h | -K | -k | -V
usage: sudo -v [-ABkNnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user] [-u
            user] [command [arg ...]]
usage: sudo [-ABbEHkNnPS] [-r role] [-t type] [-C num] [-D directory] [-g
            group] [-h host] [-p prompt] [-R directory] [-T timeout] [-u
            user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkNnS] [-r role] [-t type] [-C num] [-D directory] [-g
            group] [-h host] [-p prompt] [-R directory] [-T timeout] [-u
            user] file ...
sousou@projet:~$ sudo python3 audit.py
python3: can't open file '/home/sousou/audit.py': [Errno 2] No such file o
r directory
sousou@projet:~$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.11.2-1+b1).
python3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
sousou@projet:~$ sudo python3 audit.py
python3: can't open file '/home/sousou/audit.py': [Errno 2] No such file o
r directory
```

Figure 16. Résolution de dépendance : l'image Debian Netinst étant minimaliste, l'interpréteur Python n'était pas présent par défaut. Installation via APT avant l'exécution du script.

Le script exploite la bibliothèque native `subprocess` pour interroger le système et exporte les résultats en JSON et CSV.

```
sousou@projet:~$ pwd
/home/sousou
sousou@projet:~$ ls
audit.py          Desktop    Downloads  Pictures  Templates
audit.pypython3   Documents  Music      Public    Videos
sousou@projet:~$ sudo python3 audit.py

=== AEGIS - Audit de Sécurité ===


--- DATE ---
2026-04-15 16:36:03.215581

--- SSH_CONFIG ---
PermitRootLogin no
PasswordAuthentication no
# PasswordAuthentication no
# the setting of "PermitRootLogin no
# PAM authentication, then enable this but set PasswordAuthentication no

--- UFW_STATUS ---
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
```

Figure 17. Sortie du script `audit.py` certifiant la conformité de l'infrastructure : `PermitRootLogin` désactivé, pare-feu UFW actif avec politique `deny` en entrée.

7. Tableau de Vulnérabilités Synthétique

aegislightblue **Vulnérabilité**	**Criticité**	**Mesure appliquée**
Accès `root` direct autorisé	**Critique**	`PermitRootLogin no` dans `sshd_config`
Authentification SSH par mot de passe	**Élevée**	`PasswordAuthentication no` – clés SSH imposées
Aucun pare-feu actif	**Élevée**	UFW configuré – `deny` par défaut – ports 22/80/443 uniquement
Pas de protection bruteforce	**Élevée**	Fail2ban installé et actif – jail SSH configurée

Table 1. Tableau synthétique des vulnérabilités identifiées et mesures appliquées

8. Conclusion et Conformité RGPD

L'infrastructure TechSud est passée d'un état critique à une architecture résiliente, modulaire et auditable. Les mesures déployées répondent directement aux obligations de l'article 32 du RGPD (Règlement UE 2016/679) relatif à la sécurité du traitement :

Bilan de conformité RGPD – Article 32

- **Chiffrement des données en transit :** assuré par OpenSSH (clés asymétriques Ed25519).
- **Contrôle d'accès strict :** désactivation de `root`, authentification par clé SSH exclusivement.
- **Résistance aux attaques :** mitigation des scans réseau via UFW et blocage du bruteforce via Fail2ban.
- **Traçabilité :** logs centralisés et rapports d'audit horodatés (`.json`/`.csv`).

L'intégralité des livrables techniques attendus pour ce module a été complétée et validée par des preuves visuelles et les scripts d'audit automatisés hébergés sur GitHub (`Aegis-groupeX`).